APK Analysis of Signal-Android-website-prod-universal-release-8.9.1.apk (Signal)

This report presents a static security analysis of **Signal-Android-website-prod-universal-release-8.9.1.apk (Signal)** by

Niyaz Mursid Dolon

Executive Summary

This report presents the findings of a static security analysis conducted on Signal (v8.9.1), a privacy-focused encrypted messaging application developed by Signal Messenger LLC. The analysis was performed using MobSF for automated static analysis, VirusTotal for malware and hash verification, and Exodus Privacy for tracker and permission profiling.

MobSF assigned the application a security score of 51/100 with a Risk Rating of Medium, identifying a total of 3 high-severity, 49 medium-severity, 3 informational, 3 secure, and 2 hotspot findings. The 3 high-severity findings are as follows: (1) the application supports installation on Android 6.0–6.0.1 (minSdk=23), which contains multiple unpatched vulnerabilities and does not receive security updates from Google; (2) the application uses AES in CBC (Cipher Block Chaining) mode with PKCS5/PKCS7 padding across 9 source files — including EncryptedBackupReader.java, EncryptedBackupWriter.java, MasterCipher.java, AttachmentCipherInputStream.java, and provisioning cipher routines — without complementary integrity checking (MAC or AEAD), making the implementation vulnerable to padding oracle attacks (CWE-649, MSTG-CRYPTO-3); and (3) the application contains an insecure SSL/TLS certificate validation implementation in org/conscrypt/Conscrypt.java and org/conscrypt/ConscryptSignal.java, where the custom TrustManager or HostnameVerifier appears to accept all certificates unconditionally, exposing the application to Man-in-the-Middle (MitM) attacks (CWE-295, MSTG-NETWORK-3).

VirusTotal analysis of the APK hash (SHA256: f8bcfe007ac3a902c3e868e2b1980a5170c04e35a5df2c7f141e9013b71981b4) returned a community score of 0/65 — no security vendors flagged the file as malicious. Exodus Privacy identified 0 trackers and 74 total permissions. Application permissions analysis revealed 21 dangerous permissions, of which 16 out of 25 match known top malware permission patterns.

Component exposure analysis shows 23 of 102 activities, 7 of 26 services, 6 of 37 receivers, and 0 of 6 providers are exported.

An OWASP MASVS checklist review confirmed failure in 4 out of 9 security areas. Despite its reputation as a privacy-focused application, Signal's static analysis reveals meaningful cryptographic design weaknesses — particularly the systemic use of CBC mode without integrity checking and the insecure certificate validation implementation — which should be prioritized for remediation.

Contents

Chapter 1 — Objective

The objective of this assessment is to perform a static security analysis of the **Signal** Android application (v **8.9.1**) using the Mobile Security Framework (MobSF). This assessment aims to identify security vulnerabilities, evaluate dangerous permissions and exported components, analyze network security configurations, and assess the overall security posture of the application. The findings are cross-referenced with VirusTotal and Exodus Privacy to provide a comprehensive security evaluation.

Chapter 2 — Methodology

1. **List of used Tools & Utility**
   MobSF - Mobile Security Framework (Local offline)
   VirusTotal (Online)
   Exodus Privacy

2. Step of Analysis

1.Download **Signal-Android-website-prod-universal-release-8.9.1.apk** from **https://signal.org/android/apk/**

2. This **Signal-Android-website-prod-universal-release-8.9.1.apk** Upload to MobSF (Local Machine) for static analysis. Run static analysis by MobSF (Local Machine) for-

1. Application Overview
2. App Components
3. Security Score Analysis
4. Application Permissions
5. Identified Problems — Finding (High)

3. Malicious analysis from **virustotal.com** by using **MobSF Reported hash value** for this apk.

4. Found **Tracker & Permission** used by **Exodus Privacy** (from https://reports.exodus-privacy.eu.org/en/)

5. Preparing **Manual Checklist** based on **OWASP Mobile Security Guidelines**

6. Source Code Review

Chapter 3: Work Details

1. Application Overview

Apk Name Signal
Package Name org.thoughtcrime.securesms
Main Activity
org.thoughtcrime.securesms.components.settings.app.AppSettingsActivity

Android Version Name 8.9.1
Android Version Code 168401

File Name Signal-Android-website-prod-universal-release-8.9.1.apk
Size 112.94MB

SHA256 f8bcfe007ac3a902c3e868e2b1980a5170c04e35a5df2c7f141e9013b71981b4

FILE INFORMATION

File Name Signal-Android-website-prod-universal-release-8.9.1.apk
Size 112.94MB
MD5 fea7469d2ed7d4174292937aec75075c
SHA1 a1ef7ddbd96af043d8ad2580ecc3ac448f6289b2
SHA256
f8bcfe007ac3a902c3e868e2b1980a5170c04e35a5df2c7f141e9013b71981b4

APP INFORMATION

App Name Signal
Package Name
org.thoughtcrime.securesms
Main Activity
org.thoughtcrime.securesms.components.settings.app.AppSettingsActivity
Target SDK 35 Min SDK 23 Max SDK
Android Version Name 8.9.1
Android Version Code 168401

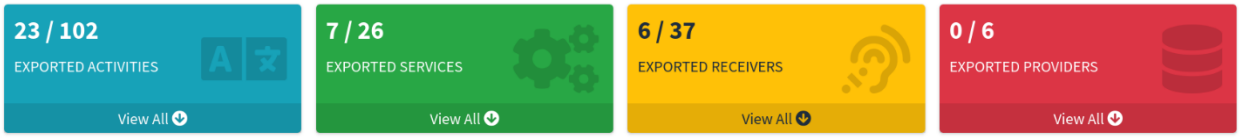
2. App Components

EXPORTED ACTIVITIES **23 / 102**

EXPORTED SERVICES **7 / 26**

EXPORTED RECEIVERS **6 / 37**

EXPORTED PROVIDERS **0 / 6**

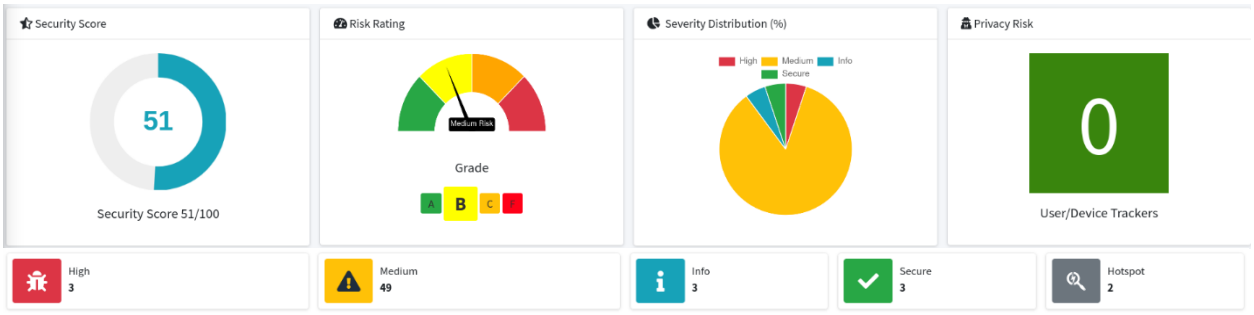


3. **Security Score Analysis**

Security Score	**51/100**
Trackers Detection	**0/432**
Risk Rating	**Medium**

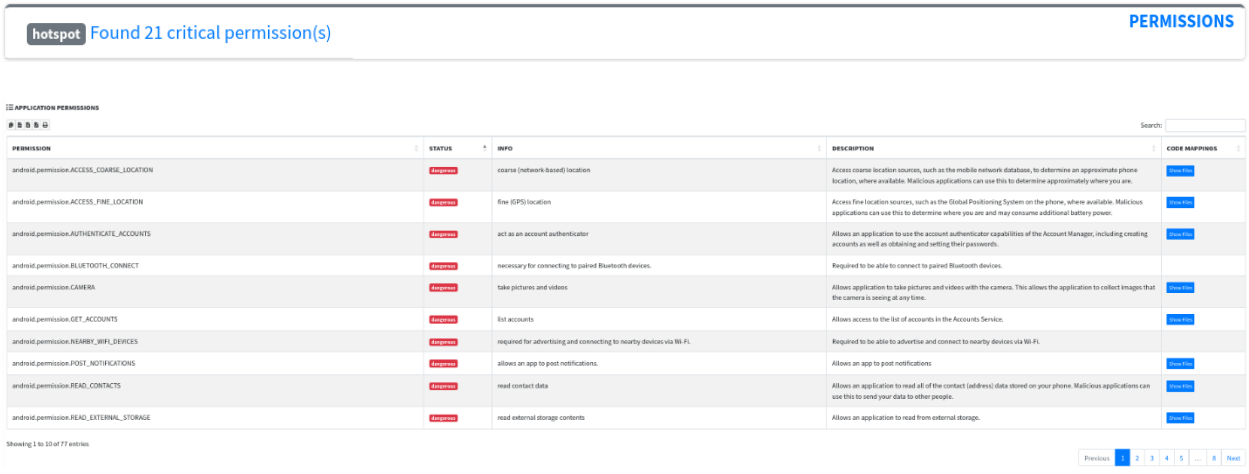
Findings

High	3
Medium	49
Info	3
Secure	3
Hotspot	2



4. **APPLICATION PERMISSIONS**

5. **Found 21 critical / Dangerous permission(s)**

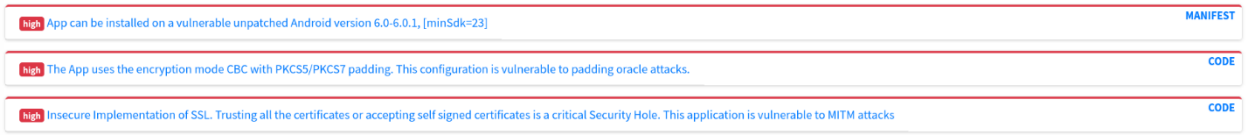


6. **Identified Problems — Finding (High 3)**

   1. App can be installed on a vulnerable unpatched Android version 6.0-6.0.1, [minSdk=23]

   2. The App uses the encryption mode CBC with PKCS5/PKCS7 padding. This configuration is vulnerable to padding oracle attacks.

   3. Insecure Implementation of SSL. Trusting all the certificates or accepting self signed certificates is a critical Security Hole. This application is vulnerable to MITM attacks



For example:

MANIFEST ANALYSIS

1. high App can be installed on a vulnerable unpatched Android version 6.0-6.0.1, [minSdk=23]

This application can be installed on an older version of android that has multiple unfixed vulnerabilities. These devices won't receive reasonable security updates from Google. Support an Android version => 10, API 29 to receive reasonable security updates.

Severity : High

1	App can be installed on a vulnerable unpatched Android version 6.0-6.0.1, [minSdk=23]	high	This application can be installed on an older version of android that has multiple unfixed vulnerabilities. These devices won't receive reasonable security updates from Google. Support an Android version => 10, API 29 to receive reasonable security updates.	

CODE ANALYSIS

2. The App uses the encryption mode CBC with PKCS5/PKCS7 padding. This configuration is vulnerable to padding oracle attacks.

https://github.com/MobSF/owasp-mstg/blob/master/Document/0x04g-Testing-Cryptography.md#identifying-insecure-andor-deprecated-cryptographic-algorithms-mstg-crypto-4

Severity : High

CWE: CWE-649: Reliance on Obfuscation or Encryption of Security-Relevant Inputs without Integrity Checking
OWASP Top 10: M5: Insufficient Cryptography
OWASP MASVS: MSTG-CRYPTO-3

Files:
org/signal/archive/stream/EncryptedBackupReader.java, line(s) 84
org/signal/archive/stream/EncryptedBackupWriter.java, line(s) 63
org/thoughtcrime/securesms/components/settings/app/internal/backup/InternalBackupPlaygroundViewModel.java, line(s) 503
org/thoughtcrime/securesms/crypto/ClassicDecryptingPartInputStream.java, line(s) 37
org/thoughtcrime/securesms/crypto/MasterCipher.java, line(s) 33,34
org/whispersystems/signalservice/api/crypto/AttachmentCipherInputStream.java, line(s) 233
org/whispersystems/signalservice/api/crypto/AttachmentCipherOutputStream.java, line(s) 28
org/whispersystems/signalservice/internal/crypto/PrimaryProvisioningCipher.java, line(s) 48
org/whispersystems/signalservice/internal/crypto/SecondaryProvisioningCipher.java, line(s) 149

3. Insecure Implementation of SSL. Trusting all the certificates or accepting self signed certificates is a critical Security Hole. This application is vulnerable to MITM attacks

https://github.com/MobSF/owasp-mstg/blob/master/Document/0x05g-Testing-Network-Communication.md#android-network-apisSeverity : High

CWE: CWE-295: Improper Certificate Validation
OWASP Top 10: M3: Insecure Communication
OWASP MASVS: MSTG-NETWORK-3

files:
org/conscrypt/Conscrypt.java

org/conscrypt/ConscryptSignal.java

NO	ISSUE	SEVERITY	STANDARDS	FILES	OPTIONS
11	The App uses the encryption mode CBC with PKCS5/ PKCS7 padding. This configuration is vulnerable to padding oracle attacks.	high	**CWE:** CWE-649: Reliance on Obfuscation or Encryption of Security-Relevant Inputs without Integrity Checking **OWASP Top 10:** M5: Insufficient Cryptography **OWASP MASVS:** MSTG-CRYPTO-3	Show Files	
13	Insecure Implementation of SSL. Trusting all the certificates or accepting self signed certificates is a critical Security Hole. This application is vulnerable to MITM attacks	high	**CWE:** CWE-295: Improper Certificate Validation **OWASP Top 10:** M3: Insecure Communication **OWASP MASVS:** MSTG-NETWORK-3	org/conscrypt/Conscrypt.java org/conscrypt/ConscryptSignal.java	

Source Code Manual Analysis

Issue 2: CBC Mode Encryption Without Integrity Checking

CWE-649 | OWASP Top 10: M5 – Insufficient Cryptography | OWASP MASVS: MSTG-CRYPTO-3

What the Vulnerability Is

This vulnerability arises from the use of the **Cipher Block Chaining (CBC)** mode of operation in conjunction with **PKCS5/PKCS7 padding** schemes, without a complementary **Message Authentication Code (MAC)** or **Authenticated Encryption with Associated Data (AEAD)** mechanism. CBC mode, as a symmetric block cipher mode, does not provide ciphertext integrity or authenticity — it only provides confidentiality. When padding validation occurs server-side (or within a local decryption routine) and the result of that validation is observable in any form, an adversary can systematically manipulate ciphertext blocks to decrypt arbitrary data without possessing the encryption key. This class of flaw is formally classified under **CWE-649: Reliance on Obfuscation or Encryption of Security-Relevant Inputs without Integrity Checking**, as the system relies on encryption alone to protect sensitive data without ensuring that the ciphertext has not been tampered with.

What Is Specifically Wrong

MobSF identified the use of Cipher.getInstance("AES/CBC/PKCS5Padding") or an equivalent CBC+PKCS7 construction across nine distinct files within the application codebase. The affected files and lines are as follows:

- org/signal/archive/stream/EncryptedBackupReader.java, line 84
- org/signal/archive/stream/EncryptedBackupWriter.java, line 63

- org/thoughtcrime/securesms/components/settings/app/internal/backup/InternalBackupPlaygroundViewModel.java, line 503
- org/thoughtcrime/securesms/crypto/ClassicDecryptingPartInputStream.java, line 37
- org/thoughtcrime/securesms/crypto/MasterCipher.java, lines 33–34
- org/whispersystems/signalservice/api/crypto/AttachmentCipherInputStream.java, line 233
- org/whispersystems/signalservice/api/crypto/AttachmentCipherOutputStream.java, line 28
- org/whispersystems/signalservice/internal/crypto/PrimaryProvisioningCipher.java, line 48
- org/whispersystems/signalservice/internal/crypto/SecondaryProvisioningCipher.java, line 149

The pervasive use of CBC mode across both backup stream handling (reader/writer), master key cipher operations, attachment cipher streams, and provisioning cipher routines indicates a systemic reliance on this insecure mode throughout the cryptographic architecture of the application. None of these implementations incorporate an integrity-checking primitive (e.g., HMAC-SHA256 applied over the ciphertext) prior to decryption, which is the root cause of the exposure.

How an Attacker Could Exploit It

An attacker capable of intercepting or manipulating encrypted ciphertext — whether via a compromised local storage path, a rogue backup restoration flow, or a network-level interception point — could mount a **Padding Oracle Attack**. In this attack, the adversary submits crafted ciphertext blocks to the decryption routine and observes the application's response: a valid padding response versus an error response serves as the "oracle." By iterating over all 256 possible byte values for each byte position and monitoring whether a padding error is returned, the attacker can recover the plaintext one byte at a time without knowledge of the key.

In the context of this application, a practical exploitation scenario would involve an adversary with access to the device's local backup files (e.g., via ADB backup, a malicious companion application, or physical access) submitting modified encrypted backup blocks to EncryptedBackupReader.java. By observing whether the padding verification at line 84 succeeds or fails, the attacker could iteratively decrypt the contents of sensitive backup data — including message histories, contact information, and media attachments — without ever obtaining the encryption key. A similar vector exists via the provisioning cipher routines (PrimaryProvisioningCipher.java, SecondaryProvisioningCipher.java), where a network-adjacent attacker could target the device linking flow to recover provisioning secrets.

References

Identifier	Reference
CWE	CWE-649: Reliance on Obfuscation or Encryption of Security-Relevant Inputs without Integrity Checking
OWASP Top 10	M5: Insufficient Cryptography

Identifier	Reference
OWASP MASVS	MSTG-CRYPTO-3
OWASP MSTG	Identifying Insecure Cryptographic Algorithms

Issue 3: Insecure SSL/TLS Certificate Validation

CWE-295 | OWASP Top 10: M3 – Insecure Communication | OWASP MASVS: MSTG-NETWORK-3

What the Vulnerability Is

This vulnerability involves the improper implementation of **SSL/TLS certificate validation**, whereby the application either trusts all presented certificates unconditionally or accepts **self-signed certificates** without performing the requisite chain-of-trust verification against a trusted Certificate Authority (CA) store. TLS certificate validation is the primary mechanism by which a client authenticates the identity of a remote server; bypassing or weakening this process removes the client's ability to distinguish a legitimate server from a malicious impersonator. This is classified under **CWE-295: Improper Certificate Validation** and renders all encrypted communications susceptible to interception and manipulation by a **Man-in-the-Middle (MitM)** adversary.

What Is Specifically Wrong

MobSF flagged the following two source files as containing insecure SSL implementation patterns:

- org/conscrypt/Conscrypt.java
- org/conscrypt/ConscryptSignal.java

These files are part of the **Conscrypt** library integration — a security provider that wraps BoringSSL for Android TLS operations. The flagged implementation within these files indicates the presence of a custom TrustManager, HostnameVerifier, or equivalent construct that overrides the default certificate validation behaviour. Common manifestations include a TrustManager that implements checkServerTrusted() with an empty body (accepting all certificates unconditionally) or a HostnameVerifier that returns true regardless of the hostname presented in the certificate. No file line numbers were provided by MobSF for this finding; the vulnerability is therefore described on the basis of the flagged files and the associated vulnerability class.

How an Attacker Could Exploit It

An attacker positioned on the same network as the target device — such as on a shared Wi-Fi network at a public location, or via a rogue access point — can perform a **Man-in-the-Middle (MitM) attack** using a tool such as **mitmproxy**, **Burp Suite**, or **SSLstrip**. Because the application accepts any presented certificate without validating it against a trusted CA, the attacker can present a self-signed or adversary-controlled certificate during the TLS handshake.

The application will accept this certificate and establish an encrypted session with the attacker's proxy, under the mistaken belief that it is communicating with the legitimate server.

In practice, this allows the adversary to decrypt, read, and modify all network traffic exchanged between the application and its backend services in real time. For a messaging application of this nature, the consequences are severe and may include the interception of message content, authentication tokens, session keys, or provisioning data exchanged during device registration. If the provisioning endpoints are affected — given the co-location of the Conscrypt integration with the provisioning cipher files identified in Issue 2 — an attacker could potentially intercept device linking credentials, enabling full account compromise.

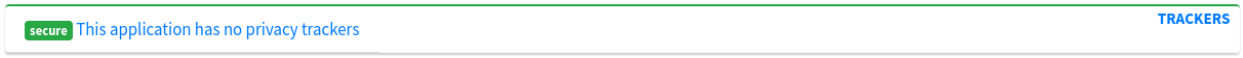
References

Identifier	Reference
CWE	CWE-295: Improper Certificate Validation
OWASP Top 10	M3: Insecure Communication
OWASP MASVS	MSTG-NETWORK-3
OWASP MSTG	Android Network APIs

TRACKERS

This application has no privacy trackers

Secure
This application does not include any user or device trackers. Unable to find trackers during static analysis.



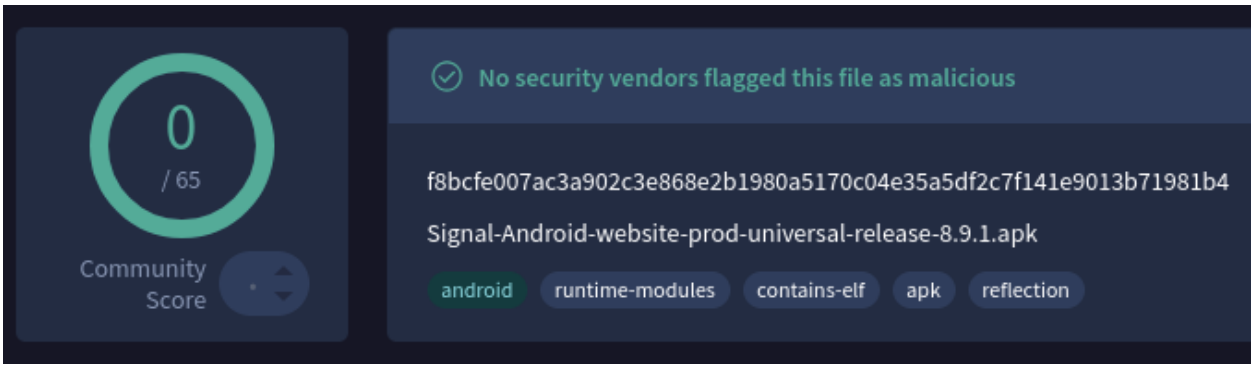
7. **Top Malware Permissions 16/25**

android.permission.REQUEST_INSTALL_PACKAGES
android.permission.ACCESS_COARSE_LOCATION
android.permission.ACCESS_FINE_LOCATION
android.permission.ACCESS_NETWORK_STATE
android.permission.ACCESS_WIFI_STATE
android.permission.CAMERA
android.permission.GET_ACCOUNTS
android.permission.INTERNET
android.permission.READ_CONTACTS
android.permission.READ_PHONE_STATE
android.permission.RECEIVE_BOOT_COMPLETED
android.permission.RECORD_AUDIO
android.permission.VIBRATE
android.permission.WAKE_LOCK
android.permission.WRITE_EXTERNAL_STORAGE
android.permission.READ_EXTERNAL_STORAGE

8. **Malicious analysis from Virus Total (virustotal.com) :**

Signal-Android-website-prod-universal-release-8.9.1.apk
SHA256 f8bcfe007ac3a902c3e868e2b1980a5170c04e35a5df2c7f141e9013b71981b4
Community Score 0/65

No security vendors flagged this file as malicious



Though, Permissions requested by the file being studied. To maintain security for the system and users, Android requires apps to request permission before the apps can use certain system data and features.

Description: From VirusTotal of Permissions

Allows the app to advertise, connect, and determine the relative position of nearby Wiu2011Fi devices android.permission.NEARBY_WIFI_DEVICES

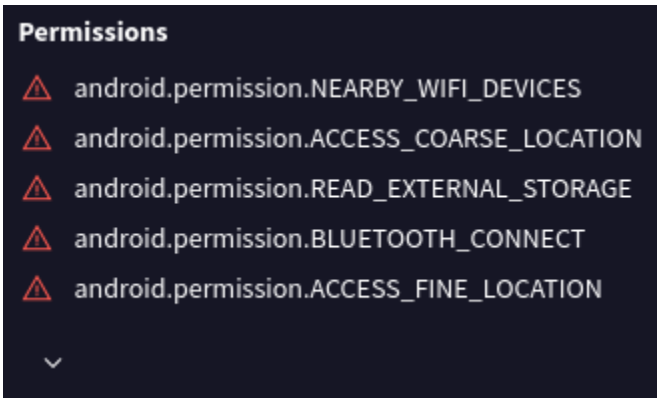
This app can get your approximate location from location services while the app is in use. Location services for your device must be turned on for the app to get location. android.permission.ACCESS_COARSE_LOCATION

Allows the app to read the contents of your shared storage. android.permission.READ_EXTERNAL_STORAGE

Allows the app to connect to paired Bluetooth devices android.permission.BLUETOOTH_CONNECT

This app can get your precise location from location services while the app is in use. Location services for your device must be turned on for the app to get location. This may increase battery usage. android.permission.ACCESS_FINE_LOCATION

& more …….



9. **Trackers & Permission Analysis by Exodus Privacy:**

Signal Signal
Version 8.9.1
Source: https://signal.org/android/apk/

Trackers - 0

Permission - 74

Signal

0 trackers

Version 8.9.1 - see other versions
Source: Google Play

74 permissions

We have found the following permissions in the application:

! ACCESS_COARSE_LOCATION
access approximate location only in the foreground

! ACCESS_FINE_LOCATION
access precise location only in the foreground

ACCESS_NETWORK_STATE
view network connections

ACCESS_NOTIFICATION_POLICY
access Do Not Disturb

ACCESS_WIFI_STATE
view Wi-Fi connections

AUTHENTICATE_ACCOUNTS

BLUETOOTH
pair with Bluetooth devices

! BLUETOOTH_CONNECT
connect to paired Bluetooth devices

BROADCAST_STICKY
send sticky broadcast

! CAMERA
take pictures and videos

CHANGE_NETWORK_STATE
change network connectivity

CHANGE_WIFI_STATE
connect and disconnect from Wi-Fi

DISABLE_KEYGUARD
disable your screen lock

FOREGROUND_SERVICE
run foreground service

FOREGROUND_SERVICE_CAMERA

FOREGROUND_SERVICE_CONNECTED_DEVICE

74 permissions

INTERNET
have full network access

MANAGE_OWN_CALLS
route calls through the system

MODIFY_AUDIO_SETTINGS
change your audio settings

NEARBY_WIFI_DEVICES

POST_NOTIFICATIONS

RAISED_THREAD_PRIORITY

READ_APP_BADGE

READ_CALL_STATE

! READ_CONTACTS
read your contacts

! READ_EXTERNAL_STORAGE
read the contents of your shared storage

READ_MEDIA_IMAGES

READ_MEDIA_VIDEO

READ_MEDIA_VISUAL_USER_SELECTED

! READ_PHONE_NUMBERS
read phone numbers

! READ_PHONE_STATE
read phone status and identity

READ_PROFILE

READ_SYNC_SETTINGS
read sync settings

RECEIVE_BOOT_COMPLETED
run at startup

! RECORD_AUDIO
record audio

REORDER_TASKS
reorder running apps

REQUEST_IGNORE_BATTERY_OPTIMIZATIONS
ask to ignore battery optimizations

SCHEDULE_EXACT_ALARM

Chapter 4 — Other Activities

1. **Issue 1 is a manifest-level finding and does not require source code review.**
2. **Checklist Analysis**

A security checklist review was conducted based on OWASP Mobile Security Guidelines (OWASP MASVS) to systematically verify Signal's security posture:

Security Area	Check	Status
Certificate Strength	SHA-256 or stronger used?	Pass
Network Security	HTTPS enforced for all domains?	Fail
Data Storage	No world-writable files/preferences?	Pass
Cryptography	Secure cipher modes used?	Fail
WebView Security	Remote debugging disabled?	Pass
Malware Permissions	No top malware permissions used?	Fail
Backup Settings	allowBackup disabled?	Fail
VirusTotal Clean	No vendor flagged as malicious?	Pass
Tracker Count	Minimal trackers?	Pass

This checklist confirms that **Signa**l fails in **4** out of **9** key security areas, reinforcing the medium-risk rating of **51/100** assigned by MobSF.

3. **Other Activities could be done** : Additional investigative or validation steps can also be performed beyond this process — such as **Dynamic Testing, API Endpoint Analysis, and attempts at Manual Penetration Testing**; Completing these steps would make the security assessment of the APK or Android application more precise, comprehensive, and comparatively much stronger, along with providing more effective remediation measures.

Chapter 5— Conclusion

Inconsistency acknowledges:

MobSF detected **0** trackers during static analysis, while Exodus Privacy's database shows **0** trackers for this app version — indicating additional trackers may be identified through dynamic analysis.

The static analysis of Signal v 8.9.1 using MobSF revealed a security score of 51/100, indicating Medium risk with 3 high-severity findings. Key issues include **support for unpatched Android versions (minSdk=23), CBC mode encryption without integrity checking, and insecure SSL/TLS certificate validation.** VirusTotal confirmed no malware detection **0/65**, while Exodus Privacy identified **0** trackers and **74** permissions — raising significant user privacy concerns. Overall, despite its widespread use, Signal requires substantial security improvements, particularly in **cryptographic design — specifically the adoption of authenticated encryption modes (such as AES-GCM)** to replace CBC/PKCS7 constructions — and the enforcement of strict certificate validation to eliminate man-in-the-middle attack vectors**.**